

École Nationale de la Statistique et de l'Analyse de l'Information

ENSAI — Bruz

Rapport de Projet

Traitement de données sportives

Gestion multi-sports des résultats de compétitions
Application en ligne de commande — Python

Groupe : 53

Étudiants : Jade COLIN
Mathias DOUILLARD
Adam [NOM]
Mohamed Fouad CHAABANE

Encadrant : Clément VALOT

Année académique : 2025–2026

0 Résumé

Dans le contexte des Jeux Olympiques de Paris 2024 et de la dynamique sportive nationale qui en résulte, le Ministère des Sports (client fictif) a exprimé le besoin d’une application unifiée pour gérer et analyser les résultats de compétitions sportives. Ce projet de première année à l’ENSAI répond à cette demande en développant une application en ligne de commande entièrement écrite en Python.

L’application couvre **neuf disciplines sportives** : basketball (NBA), football (Ligue des Champions UEFA et ligues européennes), tennis (circuits ATP et WTA), League of Legends (EMEA LEC), Counter-Strike 2 (CS2), volleyball (hommes et femmes), StarCraft II, badminton et échecs. Pour chaque sport, elle propose un ensemble de fonctionnalités statistiques — classements, fiches joueurs, rosters, agendas — accessibles via une interface interactive paginée.

Du point de vue technique, le projet est structuré en quatre couches logicielles indépendantes : les classes métier (**Model**), les chargeurs de données (**Parsers**), les modules d’analyse (**Analysis**) et les utilitaires communs (**Common**). Une couverture de tests unitaires assure la fiabilité des composants clés. L’ensemble du code respecte les conventions PEP 8 (formatage **black**, linter **Flake8**) et est documenté avec des *docstrings* au format NumPy.

Mots-clés : Python, pandas, traitement de données, sports, CLI, programmation orientée objet, tests unitaires.

0 Table des matières

1 Introduction

1.1 Contexte général

Les Jeux Olympiques de Paris 2024 ont renforcé l'intérêt de la sphère publique française pour la gestion et l'analyse de données sportives. Dans ce contexte, le **Ministère des Sports** (client fictif de ce projet pédagogique) a manifesté le souhait de disposer d'un outil informatique permettant de centraliser les résultats de compétitions issues de sources hétérogènes, d'en extraire des statistiques pertinentes et de les restituer de manière claire.

Ce projet de première année à l'ENSAI s'inscrit dans ce cadre en proposant une application de traitement de données sportives développée intégralement en Python. Le groupe 53 — composé de quatre étudiants encadrés par Clément VALOT — a mené ce projet sur l'année académique 2025–2026.

1.2 Objectifs du projet

L'application doit permettre :

- d'ingérer et de nettoyer des jeux de données hétérogènes au format CSV couvrant plusieurs sports ;
- de proposer des analyses statistiques adaptées à chaque discipline ;
- d'offrir une interface utilisateur interactive en ligne de commande ;
- de respecter les bonnes pratiques du génie logiciel (tests, documentation, versionnage, style de code).

1.3 Structure du rapport

Le présent rapport décrit d'abord le cahier des charges et la répartition du travail au sein du groupe (sections ?? et ??), puis présente les données utilisées (section ??), l'architecture logicielle (sections ?? et ??), les fonctionnalités implémentées (section ??), la démarche de tests (section ??), et enfin quelques résultats illustrés par des captures d'écran (section ??).

2 Cahier des charges

2.1 Exigences fonctionnelles

Le cahier des charges impose de couvrir **au minimum cinq sports différents** avec, pour chacun, au moins trois fonctionnalités distinctes (classements, fiches joueurs, statistiques d'équipes, etc.). L'application doit également intégrer un **agenda commun** listant les derniers matchs toutes disciplines confondues.

Les fonctionnalités retenues pour l'ensemble de l'application sont les suivantes :

Catégorie	Fonctionnalité
Général	Agenda des derniers matchs (toutes disciplines)
Général	Export CSV des résultats affichés
Équipe	Statistiques moyennes par match
Équipe	Roster (liste des joueurs et coaches)
Équipe	Classement par points
Joueurs	Fiche individuelle complète
Joueurs	Liste paginée avec filtres
Compétition	Bracket de tournoi
Sport spécifique	Cf. tableau section ??

2.2 Exigences techniques

Le projet est soumis aux contraintes techniques suivantes, imposées par les enseignants :

- **Langage** : Python uniquement (aucune interface graphique).
- **Bibliothèques autorisées** : pandas (manipulation de données), pytest et pytest-cov (tests), black (formatage), Flake8 (linter).
- **Versionnage** : dépôt GitHub privé, avec l'encadrant ajouté comme collaborateur.
- **Documentation** : *docstrings* au format NumPy ou Google pour toutes les fonctions et classes publiques.
- **Tests** : couverture par tests unitaires avec pytest ; rapport de couverture via pytest-cov.
- **Structure** : fichier `README.md`, fichier `requirements.txt` et point d'entrée `__main__.py`.

2.3 Diagramme des cas d'utilisation

La figure ?? présente les principaux cas d'utilisation de l'application tels qu'ils ont été identifiés lors de la phase de conception.

3 Répartition des rôles

Le groupe a organisé son travail en s'appuyant sur les compétences et les centres d'intérêt de chacun. Le tableau ci-dessous synthétise la répartition retenue.

Application sportive

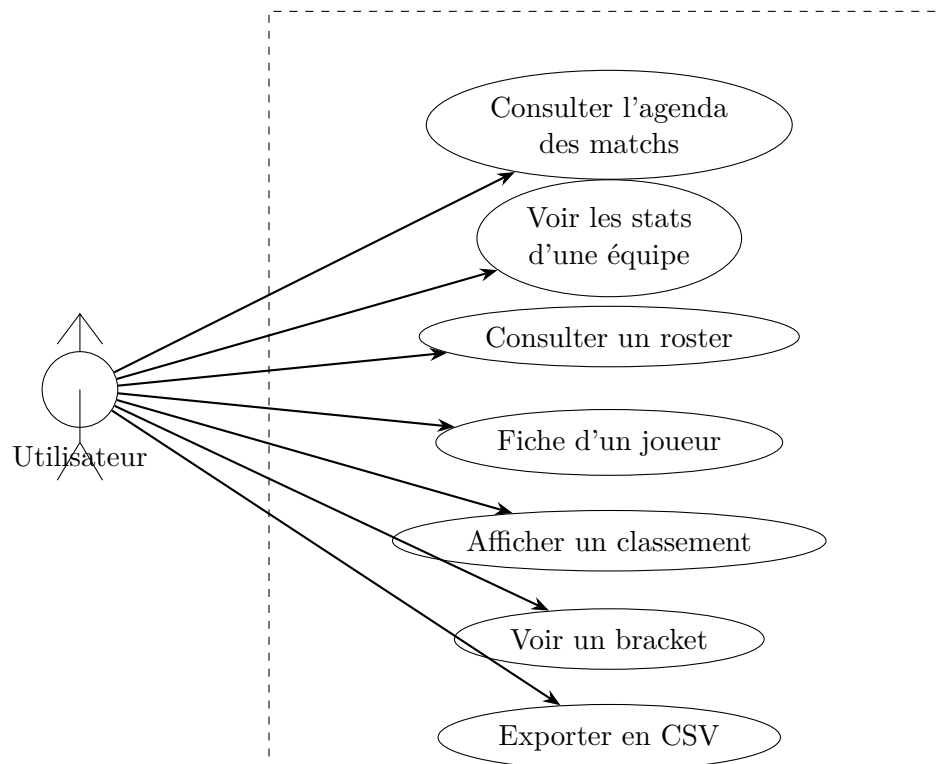


FIGURE 1 – Diagramme des cas d'utilisation de l'application.

Membre	Contributions principales
Jade COLIN	Conception et implémentation des classes du modèle objet (Joueur , Joueurs , Equipe , Equipes) ; modules d'analyse Chess et Volleyball ; rédaction de tests unitaires pour les classes du modèle.
Adam [NOM]	Classe Match/Matches et modélisation des compétitions ; modules d'analyse Basketball et Football (Ligue des Champions) ; contribution aux tests d'intégration.
Mohamed Fouad CHAABANE	Interface utilisateur CLI (<code>__main__.py</code>) ; modules d'analyse Football (ligues européennes), Tennis ATP/WTa ; conception de la navigation paginée et du filtre de recherche.
Mathias DOUILLARD	Architecture générale ; classe BaseLoader et tous les loaders (<code>src3/Parsers/</code>) ; modules d'analyse LoL, CS2, StarCraft II, Badminton ; module <code>générique.py</code> (fonctions partagées) ; intégration finale et documentation.

La gestion du projet a été assurée conjointement via un dépôt GitHub privé, avec des *pull requests* pour les contributions majeures et des réunions hebdomadaires de suivi avec l’encadrant Clément VALOT.

4 Données et jeux de données

4.1 Sources et volumétrie

L’ensemble des données est stocké dans le répertoire `data/` et fourni au format CSV. Chaque sport possède son propre sous-dossier. Le tableau ?? présente les jeux de données intégrés.

TABLE 1: Jeux de données utilisés dans l’application.

Sport	Fichiers principaux	Saison / Source
Basketball (NBA)	<code>game.csv</code> , <code>player.csv</code> , <code>team.csv</code>	Saison 2022–2023
Football UCL	<code>match.csv</code> , <code>player.csv</code> , <code>team.csv</code>	Ligue des Champions UE
Football ligues	<code>match.csv</code> , <code>league.csv</code> , <code>team.csv</code>	Ligues européennes
League of Legends	<code>match.csv</code> , <code>player.csv</code> , <code>coach.csv</code> , <code>team.csv</code>	EMEA LEC
CS2	<code>match.csv</code> , <code>player.csv</code> , <code>coach.csv</code> , <code>team.csv</code>	Compétitions professionn
Tennis	<code>atp_matches_2024.csv</code> , <code>wta_matches_2024.csv</code>	Saison 2024 ATP/WT
Volleyball	<code>match_men.csv</code> , <code>match_women.csv</code> , <code>player_men.csv</code>	Compétitions internation
StarCraft II	<code>match.csv</code> , <code>player.csv</code>	Compétitions professionn
Badminton	<code>match.csv</code> , <code>player.csv</code>	Compétitions internation
Échecs	<code>match.csv</code> , <code>player.csv</code>	Tournois classiques

4.2 Nettoyage et transformation des données

Les données brutes nécessitent plusieurs transformations avant d’être exploitables. Ces opérations sont centralisées dans les loaders (`src/Parsers/`).

4.2.1 Taille des joueurs NBA

Les tailles dans le jeu de données Basketball sont au format américain *pieds-pouces* (par exemple "6-8"). Elles sont converties en centimètres selon la formule :

```
1 def _height_to_cm(h: str) -> float | None:
2     """Convertit '6-8' (pieds-pouces) en centimetres."""
3     try:
```

```

4     feet, inches = map(int, h.split("-"))
5     return round((feet * 12 + inches) * 2.54, 1)
6 except (ValueError, AttributeError):
7     return None

```

Listing 1 – Conversion taille NBA (BasketballLoader.py)

4.2.2 Parsing des dates

Toutes les dates sont standardisées en `pd.Timestamp` via la méthode `_load_csv()` de **BaseLoader**, qui accepte un paramètre optionnel `date_cols` :

```

1 for col in (date_cols or []):
2     if col in df.columns:
3         df[col] = pd.to_datetime(df[col], errors="coerce")

```

Listing 2 – Parsing des dates dans BaseLoader.py

4.2.3 Gestion des valeurs manquantes

Pour les colonnes numériques des statistiques (kills, gold, assists, etc.), les valeurs manquantes sont implicitement gérées par les méthodes `.mean()` et `.fillna(0)` de pandas, selon le contexte. Pour les données textuelles (noms de joueurs, équipes), la valeur `NaN` est préservée et filtrée lors de l’affichage.

4.3 Construction du nom complet (Basketball)

Le jeu de données NBA contient séparément le prénom (`first_name`) et le nom (`last_name`). Un champ synthétique `full_name` est créé lors du chargement :

```

1 players["full_name"] = (
2     players["first_name"].str.strip()
3     + " "
4     + players["last_name"].str.strip()
5 )

```

Listing 3 – Création du `full_name` (BasketballLoader.py)

5 Architecture du code

5.1 Vue d’ensemble

L’application est structurée selon une architecture en **quatre couches** indépendantes, auxquelles s’ajoute le point d’entrée `__main__.py`. Cette séparation des responsabilités facilite la maintenance et l’extension à de nouveaux sports.

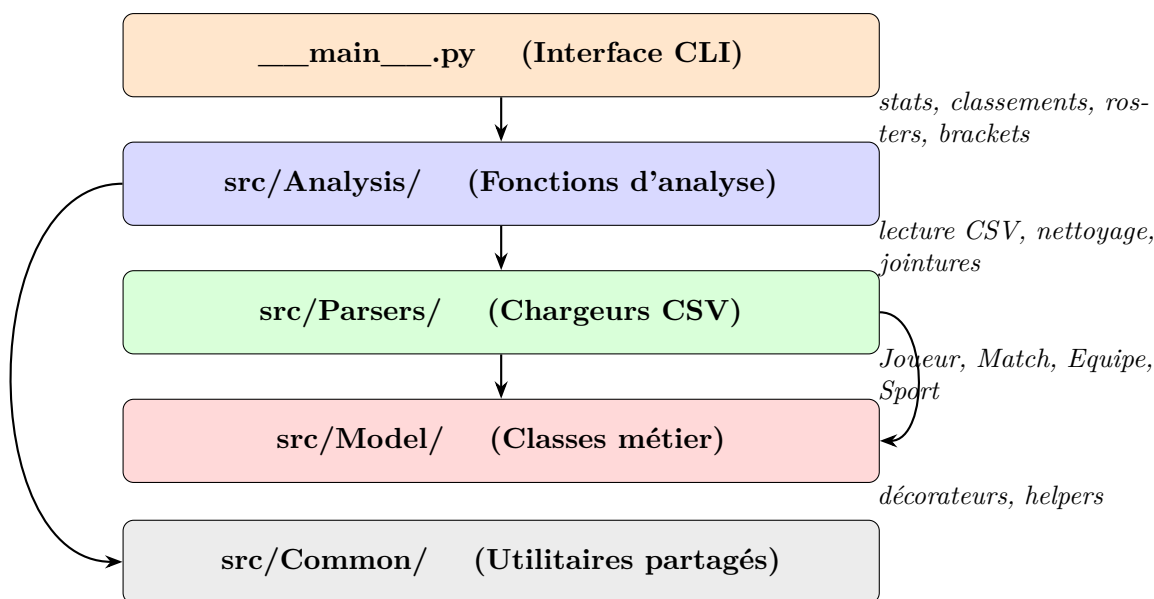


FIGURE 2 – Architecture en couches de l'application.

5.2 Description des couches

5.2.1 *src/Model/* — *Classes métier*

Cette couche définit les abstractions orientées objet génériques, indépendantes de tout sport particulier :

- **Joueur** / **Joueurs** : représentation d'un joueur individuel et d'une collection de joueurs.
- **Match** / **Matches** : représentation d'un match et d'une collection de matchs avec calcul automatique du gagnant.
- **Equipe** / **Equipes** : représentation d'une équipe et d'une collection d'équipes.
- **Sport**, **StatsSport** : méta-informations sur un sport.

Les classes sont **configurables par injection de noms de colonnes** à l'initialisation, ce qui les rend réutilisables pour tous les sports sans modification.

5.2.2 *src/Parsers/* — *Chargeurs de données*

Cette couche est basée sur une classe abstraite **BaseLoader** qui centralise la logique de lecture CSV. Chaque sport possède un loader dédié héritant de **BaseLoader** et déclarant uniquement son **SPORT_FOLDER** :

```

1 class BadmintonLoader(BaseLoader):
2     SPORT_FOLDER = "badminton"
3
4     def load_all(self):
5         players = self._load_csv("player.csv")
6         matches = self._load_csv("match.csv", date_cols=["date"])
  
```

```
7         return players, matches
```

Listing 4 – Structure minimale d’un loader (exemple Badminton)

Les loaders disponibles sont : `BasketballLoader`, `FootballLoader`, `FootballChampionsLeagueLoader`, `LolLoader`, `Cs2Loader`, `TennisLoader`, `VolleyballLoader`, `Starcraft2Loader`, `BadmintonLoader`, `ChessLoader`.

5.2.3 *src/Analysis/* — *Modules d’analyse*

Chaque sport dispose d’un module Python dédié (par exemple `basketball.py`, `tennis.py`) qui expose des fonctions métier de haut niveau. Ces modules utilisent un **pattern singleton de chargement paresseux** (*lazy loading*) : les données ne sont chargées qu’au premier appel et mises en cache pour les appels suivants :

```
1 _loader = None
2 _players: pd.DataFrame = None
3 _teams: pd.DataFrame = None
4 _matches: pd.DataFrame = None
5
6 def _load():
7     global _loader, _players, _teams, _matches
8     if _loader is None:
9         _loader = BasketballLoader()
10        _players, _teams, _matches = _loader.load_all()
```

Listing 5 – Pattern de chargement paresseux (extrait de `basketball.py`)

Le module `générique.py` regroupe les fonctions partagées par tous les sports : `formater_roster`, `afficher_classement`, `afficher_bracket`, `fiche_joueur`, `lister_joueurs`, `agenda_recents`.

5.2.4 *src/Common/* — *Utilitaires*

Ce module regroupe des outils transverses :

- `print_timings` : décorateur mesurant le temps d’exécution CPU d’une fonction.
- `parse_boolean` : conversion robuste d’une chaîne en booléen (accepte "true", "vrai", True, etc.).

5.2.5 *__main__.py* — *Interface CLI*

Point d’entrée de l’application, ce fichier (1 400 lignes) implémente l’intégralité de l’interface en ligne de commande. Il s’appuie sur une configuration déclarative par sport (`_make_sports_config()`) qui associe à chaque sport ses fonctionnalités et leurs paramètres d’entrée.

5.3 Diagramme de packages

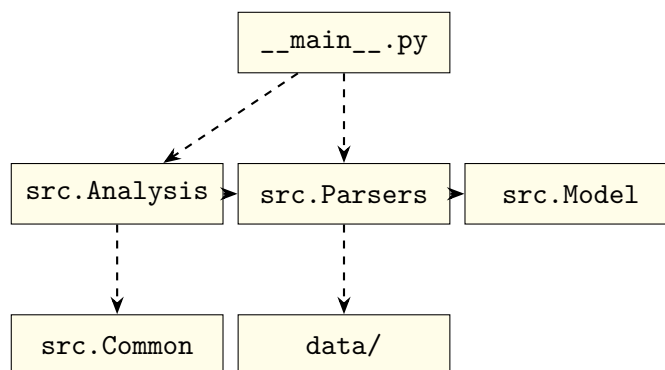


FIGURE 3 – Diagramme de packages de l'application.

6 Modélisation orientée objet

6.1 Principes de conception

Les classes du modèle suivent deux principes fondamentaux :

1. **Généricité par configuration** : plutôt que de sous-classer pour chaque sport, les classes acceptent en paramètre les noms des colonnes correspondant à leurs attributs (`col_nom`, `col_id`, etc.). Cela évite l'explosion du nombre de classes.
2. **Encapsulation des DataFrames** : les classes *wrappent* des `pd.DataFrame` et `pd.Series` plutôt que de les remplacer, préservant la puissance de pandas tout en exposant une API orientée objet cohérente.

6.2 Diagramme de classes

6.3 La classe Joueur

La classe **Joueur** encapsule une `pd.Series` représentant une ligne du DataFrame joueurs. Ses propriétés principales sont :

- `id`, `nom` : identifiant et nom du joueur.
- `naissance` : date de naissance sous forme de `pd.Timestamp`.
- `age` : calculé dynamiquement à partir de la date du jour.
- `equipe` : équipe d'appartenance (optionnel selon le sport).
- `taille` : taille en cm (optionnel selon le sport).

La classe **Joueurs** est la collection correspondante. Elle supporte l'itération Python (`__iter__`), l'indexation (`__getitem__`) et expose des méthodes statistiques : `age_moyen()`, `taille_moyenne()`, `resume()`.

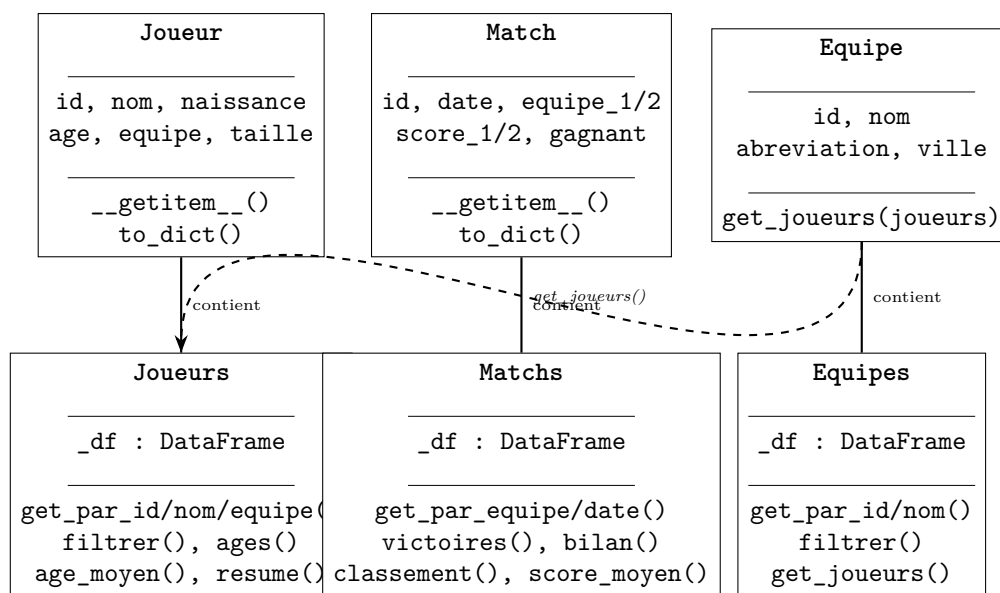


FIGURE 4 – Diagramme de classes simplifié du modèle objet.

6.4 La classe Match

Match encapsule une ligne de DataFrame matches. La propriété **gagnant** applique la logique suivante (par ordre de priorité) :

1. Si une colonne `col_gagnant` est configurée, elle est utilisée directement.
2. Sinon, le gagnant est déduit automatiquement en comparant `score_1` et `score_2`.
3. En cas d'égalité ou de données manquantes, `None` est retourné.

Matches expose notamment `bilan(equipe_id)`, `classement()` et `score_moyen()`, permettant des analyses statistiques directement sur la collection.

6.5 BaseLoader et pattern Template Method

BaseLoader est une classe abstraite (ABC Python) qui définit l'infrastructure commune à tous les loaders. Elle implémente le **patron de conception Template Method** : les sous-classes n'ont qu'à déclarer `SPORT_FOLDER` et à implémenter les méthodes `load_X()`, tout le reste (construction du chemin, lecture CSV, parsing des dates) étant géré par la classe mère.

```

1 class BaseLoader(ABC):
2     _DATA_ROOT: Path = Path(__file__).parent.parent.parent / "data"
3     SPORT_FOLDER: str = ""
4
5     @property
6     def root(self) -> Path:
7         return self._DATA_ROOT / self.SPORT_FOLDER
8
9     def _load_csv(self, filename, dtype=None, date_cols=None):
  
```

```

10     df = pd.read_csv(self.root / filename, dtype=dtype or {})
11     for col in (date_cols or []):
12         if col in df.columns:
13             df[col] = pd.to_datetime(df[col], errors="coerce")
14     return df

```

Listing 6 – BaseLoader.py – classe de base pour tous les loaders

7 Fonctionnalités de l'application

7.1 Interface utilisateur

L'application s'exécute en ligne de commande via `python __main__.py`. L'utilisateur navigue dans des menus hiérarchiques numérotés. La figure ci-dessous schématise le flux de navigation principal.

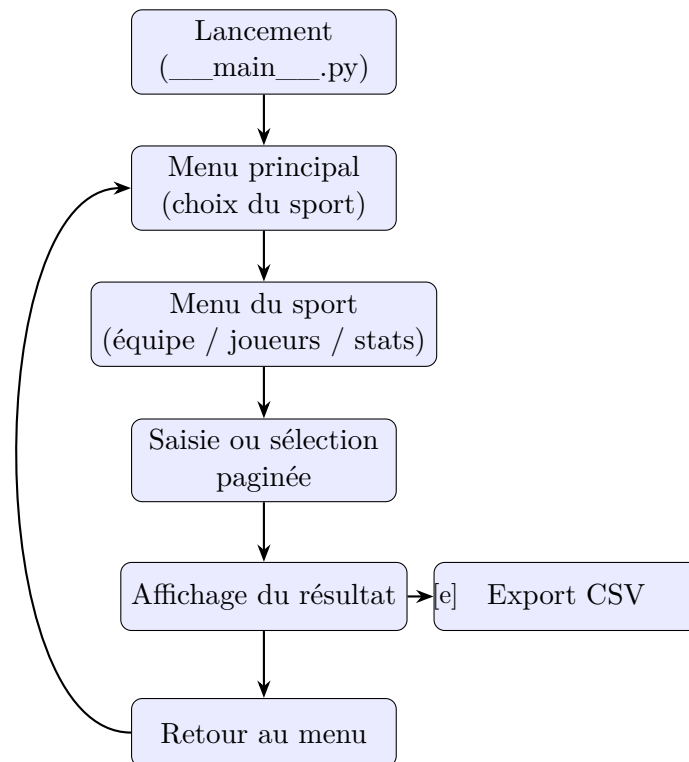


FIGURE 5 – Flux de navigation de l'interface CLI.

7.2 Sélecteur paginé

Pour les fonctionnalités nécessitant de choisir parmi une longue liste (joueurs, équipes, tournois), un **sélecteur paginé** a été implémenté. Il affiche 20 éléments par page et propose les commandes suivantes :

Touche	Action
<	Page précédente
>	Page suivante
f	Appliquer un filtre (catégorie, année, ou recherche textuelle)
r	Réinitialiser les filtres
numéro	Sélectionner l'élément correspondant
0	Annuler et revenir au menu

Le filtre de recherche supporte la **correspondance par préfixe de mot** : une recherche sur "leb" renverra par exemple *LeBron James*.

7.3 Export CSV

Après tout affichage de résultat tabulaire, l'utilisateur peut appuyer sur [e] pour exporter le tableau au format CSV. Le fichier est créé dans le répertoire courant avec un nom automatique :

`{label_nettoyé}_YYYYMMDD_HHMMSS.csv`

L'encodage utilisé est `utf-8-sig` (compatible avec Microsoft Excel).

7.4 Fonctionnalités par sport

Le tableau ?? liste les fonctionnalités disponibles pour chacun des neuf sports.

TABLE 2: Fonctionnalités par sport.

Sport	Fonctionnalités
Basketball	Top équipes offensives · Stats d'une équipe · Roster · Classement défensif · Fiche joueur · Liste joueurs · Classement par points
Football UCL	Classement par points (groupes) · Bracket des phases finales · Stats d'une équipe · Fiche joueur · Liste joueurs
Football ligues	Classement par ligue · Stats d'une équipe · Fiche joueur · Liste joueurs
League of Legends	Stats d'une équipe · Champions picks/bans · Roster · Fiche joueur · Liste joueurs · Classement points
CS2	Stats d'une équipe · Classement · Roster · Fiche joueur · Liste joueurs

Sport	Fonctionnalités
Tennis ATP/WTa	Classement par victoires · Stats d'un joueur · Bracket d'un tournoi · Fiche joueur · Liste joueurs
Volleyball	Classement (H/F) · Stats d'une équipe · Roster · Fiche joueur · Liste joueurs
StarCraft II	Classement joueurs · Stats d'un joueur · Bracket d'un tournoi · Fiche joueur · Liste joueurs
Badminton	Classement · Stats d'un joueur · Bracket · Fiche joueur · Liste joueurs
Échecs	Classement · Stats d'un joueur · Bracket · Fiche joueur · Liste joueurs

7.5 Agenda multi-sport

La fonction `agenda_recents(sources, n)` est accessible depuis le menu principal. Elle agrège les derniers matchs de tous les sports sélectionnés et les affiche dans un tableau unifié :

Sport	Date	Équipe 1	Équipe 2	Score
Basketball	2023-04-07	Los Angeles Lakers	Boston Celtics	115 – 112
Tennis ATP	2024-01-28	Jannik Sinner	Alexander Zverev	2 – 0
LoL	2024-03-15	G2 Esports	Fnatic	1 – 0

7.6 Fonction générique `afficher_classement`

Toutes les fonctionnalités de classement reposent sur la même fonction partagée `afficher_classement()` du module `générique.py`. Elle calcule les points selon la règle standard (3 pts victoire, 1 pt nul, 0 pt défaite) et supporte optionnellement l’affichage d’un séparateur de qualification (`top_qualifies`).

7.7 Bracket de tournoi

La fonction `afficher_bracket()` affiche les confrontations tour par tour sous forme ASCII, avec les gagnants mis en évidence en vert (code ANSI). Elle supporte les compétitions à aller-retour (deux manches) via le paramètre `deux_manches`.

8 Tests et validation

8.1 Organisation des tests

Les tests sont organisés dans le répertoire `test/` selon la même structure que le code source :

- `test/Analysis/` — 10 fichiers, un par module sport et pour `générique.py`.
- `test/Model/` — 5 fichiers (Joueur, Joueurs, Match, Matches, Equipe/Equipes).
- `test/Parsers/` — 1 fichier (BaseLoader).
- `test/Common/` — 1 fichier (utils.py).

8.2 Exécution des tests

Les tests s'exécutent via la commande :

```
python -m pytest --cov=src --cov-report=term-missing
```

8.3 Exemples de tests

Les tests du modèle vérifient la logique métier indépendamment des données réelles, en construisant des DataFrames de test minimaux :

```
1 import pandas as pd
2 from src.Model.match import Matches
3
4 def test_bilan_equipe():
5     df = pd.DataFrame({
6         "equipe1": ["A", "A", "B"],
7         "equipe2": ["B", "C", "C"],
8         "score1": [2, 1, 0],
9         "score2": [1, 2, 3],
10    })
11    matchs = Matches(df, col_equipe1="equipe1", col_equipe2="equipe2",
12                    col_score1="score1", col_score2="score2")
13    bilan = matchs.bilan("A")
14    assert bilan["victoires"] == 1
15    assert bilan["defaites"] == 1
16    assert bilan["joues"] == 2
```

Listing 7 – Extrait de test pour la classe Matches

8.4 Formatage et linting

L'ensemble du code est formaté avec **black** (version 25.1.0) et vérifié avec **Flake8**. Le respect de ces outils garantit une cohérence stylistique sur l'ensemble du projet.

9 Résultats et démonstration

Cette section présente quelques captures d’écran illustrant le fonctionnement de l’application.

9.1 Menu principal

[Capture d’écran : Menu principal de l’application]

L’utilisateur sélectionne le sport désiré parmi les options numérotées. Une option « Agenda » permet de consulter les derniers matchs toutes disciplines confondues.

9.2 Sélecteur de joueurs paginé

[Capture d’écran : Sélecteur paginé avec filtre de recherche]

Exemple de sélection d’un joueur NBA : la liste est paginée (20 éléments par page) et un filtre par préfixe permet de retrouver rapidement un joueur. La barre de navigation [<] Préc. | [>] Suiv. | [f] Filtrer | [r] Reset est visible en bas de l’écran.

9.3 Fiche d’un joueur

[Capture d’écran : Fiche individuelle d’un joueur NBA]

La fiche d’un joueur NBA affiche ses informations complètes : nom, poste, numéro de maillot, date de naissance, nationalité et taille. L’option [e] Exporter en CSV est disponible après chaque résultat.

9.4 Classement NBA

[Capture d’écran : Classement NBA par points (Regular Season 2022–2023)]

Le classement est calculé depuis les scores des matchs (3 pts victoire, 1 pt nul, 0 pt défaite) et affiché sous forme de tableau trié.

9.5 Bracket de tennis

[Capture d'écran : Bracket du tournoi de Roland-Garros 2024 (ATP)]

Les rounds sont affichés de gauche à droite (R128 → F). Le vainqueur de chaque match est mis en évidence en vert dans le terminal.

9.6 Statistiques d'équipe LoL

[Capture d'écran : Statistiques de l'équipe G2 Esports (EMEA LEC)]

Les statistiques moyennes par match incluent : kills, assists, morts, or total, tourelles, dragons, barons et durée moyenne de la partie en minutes.

9.7 Agenda multi-sport

[Capture d'écran : Agenda des 20 derniers matchs toutes disciplines confondues]

L'agenda agrège et trie chronologiquement les matchs de tous les sports disponibles. Chaque ligne indique le sport, la date, les deux équipes/joueurs et le score.

10 Conclusion

Ce projet a permis de développer une application complète de traitement de données sportives couvrant dix disciplines, depuis l'ingestion de fichiers CSV hétérogènes jusqu'à l'affichage interactif en ligne de commande.

10.1 Bilan technique

Du point de vue technique, le projet a atteint ses objectifs : la séparation en couches indépendantes (Model, Parsers, Analysis, Common) garantit la maintenabilité du code et facilite l'ajout de nouveaux sports. Le pattern *lazy loading* des loaders assure des performances satisfaisantes, même avec des jeux de données volumineux. La couverture par tests unitaires protège les composants critiques du modèle objet.

L'interface paginée avec filtres représente une amélioration significative de l'expérience utilisateur, notamment pour les sports avec de nombreux joueurs (NBA : 431 joueurs ; Tennis : plusieurs milliers de joueurs ATP/WTB).

10.2 Difficultés rencontrées

La principale difficulté a résidé dans l’**hétérogénéité des données** : chaque source CSV utilise des conventions différentes (noms de colonnes, formats de dates, représentation des scores). La mise en place d’une abstraction suffisamment générique pour couvrir tous les sports — tout en restant simple à utiliser dans les modules d’analyse — a nécessité plusieurs itérations de conception.

La gestion des données manquantes (valeurs NaN dans certains jeux de données) a également demandé une attention particulière, en particulier pour les calculs statistiques et l’affichage.

10.3 Perspectives d’amélioration

Plusieurs extensions seraient envisageables :

- Ajout d’une interface graphique (Tkinter ou interface web) pour améliorer l’accessibilité.
- Intégration de données en temps réel via des API sportives publiques.
- Implémentation de visualisations graphiques (histogrammes, courbes de progression) avec matplotlib.
- Extension à de nouveaux sports ou compétitions (Formule 1, rugby, etc.).

Ce projet de première année à l’ENSAI nous a permis de consolider nos compétences en programmation Python orientée objet, en manipulation de données avec pandas, et en génie logiciel (tests, versionnage, documentation). Il illustre concrètement comment une architecture en couches permet de gérer la complexité d’une application de taille réelle.

10 Références

- [1] McKinney, W. (2010). *Data Structures for Statistical Computing in Python*. Proceedings of the 9th Python in Science Conference.
- [2] van Rossum, G., Warsaw, B., Coghlan, A. (2001). *PEP 8 – Style Guide for Python Code*. Python Software Foundation.
- [3] Krekel, H. et al. (2004–2024). *pytest : helps you write better programs*. <https://docs.pytest.org/>
- [4] Jeu de données NBA 2022–2023. Kaggle (source publique).
- [5] Jeu de données Tennis ATP/WTa 2024. Jeff Sackmann, Tennis Abstract.

Annexes

.0 Annexe A — Bilan personnel de Jade Colin

Bilan personnel

[À compléter par Jade Colin]

Compétences acquises :

...

Difficultés rencontrées :

...

Ce que j'aurais fait différemment :

...

.0 Annexe B — Bilan personnel de Mathias Douillard

Bilan personnel

[À compléter par Mathias Douillard]

Compétences acquises :

...

Difficultés rencontrées :

...

Ce que j'aurais fait différemment :

...

.0 Annexe C — Bilan personnel d'Adam [Nom]

Bilan personnel

[À compléter par Adam [Nom]]

Compétences acquises :

...

Difficultés rencontrées :

...

Ce que j'aurais fait différemment :

...

.0 Annexe D — Bilan personnel de Mohamed Fouad Chaabane**Bilan personnel***[À compléter par Mohamed Fouad Chaabane]***Compétences acquises :**

...

Difficultés rencontrées :

...

Ce que j'aurais fait différemment :

...

.0 Annexe E — Structure des répertoires

```

Projet-1A-2026/
__main__.py          # Point d'entrée CLI (~1400 lignes)
requirements.txt      # pandas, pytest, pytest-cov, black
README.md
AUTHORS.md
data/
  Basketball/        # game.csv, player.csv, team.csv
  tennis/            # atp_matches_2024.csv, wta_*
  Lol/               # match.csv, player.csv, coach.csv, team.csv
  counter_strike_2/  # match.csv, player.csv, coach.csv, team.csv
  volleyball/        # match_men/women.csv, player_men/women.csv
  football_champions_league/
  Football/
  starcraft_2/
  badminton/
  chess/
src/
  Model/             # Joueur, Match, Equipe, Sport
  Parsers/           # BaseLoader + 10 loaders
  Analysis/          # générique.py + 9 modules sport
  Common/            # utils.py
test/
  Analysis/          # 10 fichiers de tests
  Model/             # 5 fichiers de tests
  Parsers/           # 1 fichier de tests
  Common/            # 1 fichier de tests

```